

OPERATING SYSTEMS

Restaurant Order Management Simulator

Project Report

Instructor	Mr. Ubaidullah
Submission Date	11 / 05 / 2026
Course	Operating Systems
Program	BS Computer Science

Group Members

#	Name	Roll No
01	Muhammad Adil Saeed	24K-0705
02	Harsh Kumar	24K-0912
03	Muhammad Haseem Samo	24K-0666

1. Summary

The Restaurant Order Management Simulator is an operating-systems-based project developed in the C programming language to simulate the workflow of a real restaurant environment. The project demonstrates the practical implementation of important operating system concepts including process synchronization, multithreading, semaphores, mutex locks, shared resources, and inter-process coordination.

The simulator manages restaurant activities such as customer order placement, chef processing, waiter coordination and kitchen scheduling through concurrent execution of multiple processes and threads. Different synchronization mechanisms were implemented to avoid race conditions and ensure efficient handling of shared resources.

The main purpose of the project is to provide a realistic simulation of restaurant operations while applying operating system principles to solve synchronization and resource management problems.

2. Introduction

Modern restaurants require efficient coordination between customers, waiters, chefs to ensure smooth operation. In real-world scenarios, multiple orders are processed simultaneously, making synchronization and proper resource allocation extremely important.

The Restaurant Order Management Simulator was developed to model these concurrent activities using operating system concepts. The project simulates multiple customers placing orders at the same time while chefs prepare food concurrently and waiters deliver completed orders. Shared resources such as kitchen stations, order queues are managed carefully using synchronization techniques.

The simulator demonstrates how operating systems handle concurrent execution, process communication, and critical section management in real-time applications. It also helps in understanding deadlock prevention, scheduling, and resource sharing in a practical environment.

Technology Stack

Layer	Technology
Programming Language	C
Concepts Used	Threads, Semaphores, Mutex, Process Synchronization
Compiler	GCC
Operating System	Linux / Ubuntu
Development Environment	g-editor

3. Project Description

The Restaurant Order Management Simulator consists of multiple modules designed to represent different restaurant operations.

3.1 Customer Module

- Simulates customers entering the restaurant
- Places food orders concurrently
- Waits for order completion
- Receives completed orders from waiters
- Generates customer request logs

3.2 Chef Module

- Processes customer orders using separate threads
- Accesses shared kitchen resources safely
- Updates order preparation status
- Prevents simultaneous conflicts during food preparation
- Simulates real-time cooking delays

3.3 Waiter Module

- Receives completed orders from chefs
 - Delivers food to customers
 - Manages serving queues
 - Coordinates between customers and kitchen staff
 - Updates order completion records
-

4. Methodology

The project was developed using a modular and structured approach where each restaurant operation was implemented and tested independently before complete system integration.

Phase	Description
Requirements Analysis	Identified restaurant workflow and synchronization requirements
System Design	Designed process flow and thread interaction structure
Thread Implementation	Created concurrent customer, chef, and waiter threads
Synchronization	Applied mutex locks and semaphores for resource protection
Testing	Performed synchronization and concurrency testing
Integration	Combined all modules into a complete simulator

Critical section problems were handled using mutex locks, while semaphores were used for controlling access to shared resources such as kitchen stations and order queues.

5. Project Implementation

The simulator is divided into several functional modules responsible for managing restaurant activities.

5.1 Order Generation Module

This module simulates customers placing orders simultaneously. Each customer request is treated as a separate thread or process to demonstrate concurrent execution.

```
static void *waiter_thread(void *arg) {
    srand((unsigned)time(NULL) ^ (unsigned)pthread_self());

    while (running && order_count < MAX_ORDERS) {
        /* Wait between orders (1-4 sec scaled by speed) */
        double delay = (1.0 + (rand() % 30) / 10.0) / sim_speed;
        usleep((useconds_t)(delay * 1000000));

        if (!running) break;

        pthread_mutex_lock(&queue_mutex);
        while (queue.size >= MAX_QUEUE && running) {
            log_event("Waiter: queue full, waiting.");
            pthread_cond_wait(&queue_not_full, &queue_mutex);
        }
        if (!running) { pthread_mutex_unlock(&queue_mutex); break; }

        pthread_mutex_lock(&state_mutex);
        if (order_count >= MAX_ORDERS) {
            pthread_mutex_unlock(&state_mutex);
            pthread_mutex_unlock(&queue_mutex);
            break;
        }

        int idx = order_count;
        Order *o = &all_orders[idx];
        memset(o, 0, sizeof(Order));
        int menu_idx = rand() % MENU_SIZE;
        o->id = next_order_id++;
        o->priority = (rand() % 5 == 0) ? 1 : 0; /* 20% VIP */
        strncpy(o->item, menu_items[menu_idx], 63);
        o->prep_time = menu_times[menu_idx];
        o->state = STATE_IN_QUEUE;
        o->created_at = time(NULL);
        o->chef_id = -1;
        order_count++;
        pthread_mutex_unlock(&state_mutex);
    }
}
```

5.2 Kitchen Processing Module

Handles food preparation operations performed by chef threads. Shared kitchen resources are protected using mutex locks to avoid resource conflicts.

```
static void *chef_thread(void *arg) {
    Chef *chef = (Chef *)arg;
    log_event("Chef %d entered the kitchen", chef->id);

    while (running) {
        /* Try to acquire kitchen semaphore (capacity constraint) */
        struct timespec ts;
        clock_gettime(CLOCK_REALTIME, &ts);
        ts.tv_sec += 1;
        if (sem_timedwait(&kitchen_sem, &ts) != 0) continue;

        pthread_mutex_lock(&queue_mutex);
        while (queue.size == 0 && running) {
            struct timespec tw;
            clock_gettime(CLOCK_REALTIME, &tw);
            tw.tv_sec += 1;
            pthread_cond_timedwait(&queue_not_empty, &queue_mutex, &tw);
        }

        if (!running) {
            pthread_mutex_unlock(&queue_mutex);
            sem_post(&kitchen_sem);
            break;
        }

        /* Check if this chef is still supposed to be active */
        if (chef->id > active_chefs) {
            pthread_mutex_unlock(&queue_mutex);
            sem_post(&kitchen_sem);
            usleep(200000);
            continue;
        }

        Order *order = pq_pop(&queue);
        if (!order) {
            pthread_mutex_unlock(&queue_mutex);
            sem_post(&kitchen_sem);
            continue;
        }

        if (order->cancelled) {
            pthread_cond_signal(&queue_not_full);
            pthread_mutex_unlock(&queue_mutex);
        }
    }
}
```

5.3 Synchronization Module

Implements semaphores and mutex mechanisms to synchronize thread execution and manage shared resources efficiently.

```
pthread_mutex_lock(&queue_mutex);
while (queue.size == 0 && running) {
    struct timespec tw;
    clock_gettime(CLOCK_REALTIME, &tw);
    tw.tv_sec += 1;
    pthread_cond_timedwait(&queue_not_empty, &queue_mutex, &tw);
}
```

Chief Thread: protects shared queue prevent race condition between chiefs

```
if (!running) break;

pthread_mutex_lock(&queue_mutex);
while (queue.size >= MAX_QUEUE && running) {
    log_event("Waiter: queue full, waiting...");
    pthread_cond_wait(&queue_not_full, &queue_mutex);
}
if (!running) { pthread_mutex_unlock(&queue_mutex); break; }

pthread_mutex_lock(&state_mutex);
```

waiter_thread :producer/waiter block when queue is full

5.4 Scheduling Module

Simulates CPU scheduling concepts by managing order execution priorities and processing sequence.

```
// Priority queue helpers //
static int pq_priority(const Order *o) {
    /* VIP=1 always beats normal=0; within same priority, older orders first */ >
    return o->priority * 1000000 + (int)(time(NULL) - o->created_at);
}
```

5.5 Logging Module

Maintains execution logs for customer orders, chef activities, waiter operations, and completed transactions.

```
static void log_event(const char *fmt, ...) {
    char buf[256];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(buf, sizeof(buf), fmt, ap);
    va_end(ap);

    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char entry[256];
    snprintf(entry, sizeof(entry), "[%02d:%02d:%02d] %s",
             t->tm_hour, t->tm_min, t->tm_sec, buf);

    pthread_mutex_lock(&log_mutex);
    int idx = (log_head + log_count) % MAX_LOG_LINES;
    strncpy(log_lines[idx], entry, 255);
    if (log_count < MAX_LOG_LINES) log_count++;
    else log_head = (log_head + 1) % MAX_LOG_LINES;
    pthread_mutex_unlock(&log_mutex);
}
```

5.6 System Implementation

The simulator uses concurrent threads to represent customers, chefs, and waiters. Shared memory structures were used for order queues, while synchronization primitives ensured safe access to critical sections.

```

int main(void) {
    signal(SIGINT, sig_handler);
    signal(SIGTERM, sig_handler);
    srand((unsigned)time(NULL));
    memset(&queue, 0, sizeof(queue));
    memset(all_orders, 0, sizeof(all_orders));
    memset(chefs, 0, sizeof(chefs));
    sem_init(&kitchen_sem, 0, active_chefs);
    printf("=== Restaurant Order Management Simulator ===\n");
    printf("Chefs: %d (dynamic), Queue capacity: %d\n", active_chefs, MAX_QUEUE);
    printf("Writing state to: %s\n", STATE_FILE);
    printf("Press Ctrl+C to stop.\n\n");
    for (int i = 0; i < MAX_CHEFS; i++) {
        chefs[i].id = i + 1;
        pthread_create(&chefs[i].thread, NULL, chef_thread, &chefs[i]);
    }
    pthread_t waiter_t, alloc_t, writer_t, cancel_t, speed_t;
    pthread_create(&waiter_t, NULL, waiter_thread, NULL);
    pthread_create(&alloc_t, NULL, allocator_thread, NULL);
    pthread_create(&writer_t, NULL, state_writer_thread, NULL);
    pthread_create(&cancel_t, NULL, cancel_watcher, NULL);
    pthread_create(&speed_t, NULL, speed_watcher, NULL);
    while (running) {
        sleep(10);
        int received = 0, in_queue = 0, preparing = 0, completed = 0, cancelled = 0;
        pthread_mutex_lock(&state_mutex);
        for (int i = 0; i < order_count; i++) {
            switch (all_orders[i].state) {
                case STATE_RECEIVED: received++; break;
                case STATE_IN_QUEUE: in_queue++; break;
                case STATE_PREPARING: preparing++; break;
                case STATE_COMPLETED: completed++; break;
                case STATE_CANCELLED: cancelled++; break;
            }
        }
        pthread_mutex_unlock(&state_mutex);
        printf("[Summary] Total=%d Queue=%d Cooking=%d Done=%d Cancelled=%d Chefs=%d\n",
            order_count, in_queue, preparing, completed, cancelled, active_chefs);
    }
    printf("\nShutting down...\n");
    pthread_cond_broadcast(&queue_not_empty);
    pthread_cond_broadcast(&queue_not_full);
    pthread_join(waiter_t, NULL);
    pthread_join(alloc_t, NULL);
    pthread_join(cancel_t, NULL);
    pthread_join(speed_t, NULL);
    for (int i = 0; i < MAX_CHEFS; i++) pthread_join(chefs[i].thread, NULL);
    pthread_join(writer_t, NULL);
    sem_destroy(&kitchen_sem);
    printf("Simulation ended. %d orders processed.\n", order_count);
    return 0;
}

```

Component	Purpose
Customer Threads	Simulate customer order requests
Chef Threads	Process and prepare food orders
Waiter Threads	Deliver completed orders
Mutex Locks	Protect critical sections
Semaphores	Control shared resource access
Shared Queues	Maintain pending orders

6. Results

The Restaurant Order Management Simulator was tested under multiple concurrent scenarios to evaluate synchronization and process coordination.

6.1 Key Outcomes

Outcome	Description
Concurrent Processing	Multiple customer orders handled simultaneously
Synchronization Efficiency	Mutex and semaphore implementation prevented race conditions
Resource Management	Shared kitchen resources managed effectively
Scheduling Simulation	Orders processed in organized execution sequence
System Stability	Simulator maintained stable execution under concurrent load
Practical Understanding	Improved understanding of operating system concepts

The simulator successfully demonstrated real-world applications of process synchronization, multithreading, and resource allocation techniques used in operating systems.

7. Conclusion

The Restaurant Order Management Simulator successfully demonstrates the implementation of core operating system concepts in a practical real-world scenario. The project applies multithreading, synchronization, semaphores, mutex locks, and resource management techniques to simulate restaurant operations efficiently.

The simulator provides a clear understanding of how concurrent processes interact and how synchronization mechanisms prevent race conditions and resource conflicts. By modeling customers, chefs, waiters as separate concurrent entities, the project reflects realistic process coordination challenges.

The project also strengthened understanding of thread management, critical section handling, scheduling, and inter-process communication. The modular design ensures maintainability and allows future improvements such as priority scheduling, deadlock handling, and advanced process communication techniques.

8. References

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- GNU Compiler Collection (GCC) Documentation. Retrieved from <https://gcc.gnu.org/>
- Linux Manual Pages. POSIX Threads Programming Documentation. Retrieved from <https://man7.org/linux/man-pages/>
- The Open Group. POSIX Standard Documentation. Retrieved from <https://pubs.opengroup.org/>
- FAST-NUCES Department of Computer Science. Operating Systems Course Notes and Lab Manuals.
- Ubuntu Linux Documentation. Retrieved from <https://ubuntu.com/server/docs>

9. Output

Terminal:

1.

```
tempname@harsh-dembla-VirtualBox:~/Downloads$ gcc -o restaurant_sim restaurant_sim.c -pthread -lm -O2
restaurant_sim.c: In function 'log_event':
restaurant_sim.c:110:54: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of s
between 218 and 245 [-Wformat-truncation=]
 110 |         snprintf(entry, sizeof(entry), "[%02d:%02d:%02d] %s",
      |                                     ^~
 111 |         t->tm_hour, t->tm_min, t->tm_sec, buf);
      |                                     ~~~~~
In file included from /usr/include/stdio.h:980,
      from restaurant_sim.c:11:
In function 'snprintf',
      inlined from 'log_event' at restaurant_sim.c:110:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 12 and 294 bytes
a destination of size 256
 54 |     return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |           ^
 55 |           __glibc_objsize (__s), __fmt,
      |           ~~~~~
 56 |           __va_arg_pack ());
      |           ~~~~~
```

//Warning does not effect the output

2.

```
tempname@harsh-dembla-VirtualBox:~/Downloads$ ./restaurant_sim
=== Restaurant Order Management Simulator ===
Chefs: 2 (dynamic), Queue capacity: 50
Writing state to: sim_state.json
Press Ctrl+C to stop.

[Summary] Total=4 Queue=2 Cooking=2 Done=0 Cancelled=0 Chefs=2
[Summary] Total=7 Queue=3 Cooking=2 Done=2 Cancelled=0 Chefs=2
[Summary] Total=13 Queue=7 Cooking=2 Done=4 Cancelled=0 Chefs=2
[Summary] Total=19 Queue=11 Cooking=2 Done=6 Cancelled=0 Chefs=2
[Summary] Total=26 Queue=15 Cooking=4 Done=7 Cancelled=0 Chefs=4
[Summary] Total=30 Queue=14 Cooking=6 Done=10 Cancelled=0 Chefs=6
[Summary] Total=35 Queue=11 Cooking=8 Done=16 Cancelled=0 Chefs=8
[Summary] Total=39 Queue=8 Cooking=8 Done=23 Cancelled=0 Chefs=8
[Summary] Total=43 Queue=6 Cooking=8 Done=29 Cancelled=0 Chefs=8
[Summary] Total=47 Queue=4 Cooking=8 Done=35 Cancelled=0 Chefs=8
[Summary] Total=50 Queue=0 Cooking=7 Done=43 Cancelled=0 Chefs=8
[Summary] Total=53 Queue=0 Cooking=5 Done=48 Cancelled=0 Chefs=6
[Summary] Total=56 Queue=0 Cooking=2 Done=54 Cancelled=0 Chefs=4
[Summary] Total=60 Queue=2 Cooking=2 Done=56 Cancelled=0 Chefs=2
[Summary] Total=63 Queue=3 Cooking=2 Done=58 Cancelled=0 Chefs=2
[Summary] Total=67 Queue=6 Cooking=2 Done=59 Cancelled=0 Chefs=2
[Summary] Total=73 Queue=11 Cooking=2 Done=60 Cancelled=0 Chefs=2
[Summary] Total=76 Queue=9 Cooking=4 Done=63 Cancelled=0 Chefs=4
[Summary] Total=82 Queue=12 Cooking=4 Done=66 Cancelled=0 Chefs=4
[Summary] Total=86 Queue=11 Cooking=6 Done=69 Cancelled=0 Chefs=6
[Summary] Total=90 Queue=8 Cooking=7 Done=75 Cancelled=0 Chefs=7
[Summary] Total=94 Queue=6 Cooking=7 Done=81 Cancelled=0 Chefs=7
[Summary] Total=99 Queue=4 Cooking=7 Done=88 Cancelled=0 Chefs=7
[Summary] Total=103 Queue=4 Cooking=7 Done=92 Cancelled=0 Chefs=7
```

3.

```

tempname@harsh-dembla-VirtualBox:~$ cd Downloads
tempname@harsh-dembla-VirtualBox:~/Downloads$ php -S localhost:8000
[Mon May 11 22:37:28 2026] PHP 8.3.6 Development Server (http://localhost:8000)
started
[Mon May 11 22:37:56 2026] 127.0.0.1:56750 Accepted
[Mon May 11 22:37:58 2026] 127.0.0.1:56750 [200]: GET /
[Mon May 11 22:37:58 2026] 127.0.0.1:56750 Closing
[Mon May 11 22:37:58 2026] 127.0.0.1:56758 Accepted
[Mon May 11 22:37:58 2026] 127.0.0.1:56758 [200]: GET /
[Mon May 11 22:37:58 2026] 127.0.0.1:56758 Closing
[Mon May 11 22:38:02 2026] 127.0.0.1:56774 Accepted
[Mon May 11 22:38:02 2026] 127.0.0.1:56774 [404]: GET /favicon.ico - No such file or directory
[Mon May 11 22:38:02 2026] 127.0.0.1:56774 Closing
[Mon May 11 22:38:02 2026] 127.0.0.1:56784 Accepted
[Mon May 11 22:38:02 2026] 127.0.0.1:56784 [200]: GET /api.php?action=status
[Mon May 11 22:38:02 2026] 127.0.0.1:56784 Closing
[Mon May 11 22:38:02 2026] 127.0.0.1:56794 Accepted
[Mon May 11 22:38:02 2026] 127.0.0.1:56794 [200]: GET /api.php?action=status
[Mon May 11 22:38:02 2026] 127.0.0.1:56794 Closing
[Mon May 11 22:38:03 2026] 127.0.0.1:56800 Accepted
[Mon May 11 22:38:03 2026] 127.0.0.1:56800 [200]: GET /api.php?action=status
[Mon May 11 22:38:03 2026] 127.0.0.1:56800 Closing
[Mon May 11 22:38:03 2026] 127.0.0.1:56802 Accepted

```

//Frontend connect through php

Platform:

RESTOSIM | ORDER_MANAGEMENT_SIMULATOR / OS_CONCEPTS_DEMO | SPEED 1x | LIVE

TOTAL ORDERS	IN QUEUE	PREPARING	COMPLETED	VIP ORDERS	ACTIVE CHEFS
39	7	8	20	3	8

KITCHEN STAFF	RECEIVED	IN QUEUE	PREPARING	COMPLETED
- Chef Marco: Pasta Carbonara (0) - Chef Yuki: Lamb Chops (8) - Chef Sofia: Lamb Chops (1) - Chef Amir: Lamb Chops (3) - Chef Léa: Margherita Pizza (2) - Chef Carlos: Pasta Carbonara (2) - Chef Priya: Chocolate Lava Cake (1) - Chef Ben: Beef Burger (1)	0	#33 Margherita Pizza (12min prep, 18s ago) #34 Lamb Chops (20min prep, 15s ago) #35 Grilled Salmon (15min prep, 13s ago) #36 Lamb Chops (20min prep, 11s ago) #37 Caesar Salad	#21 Lamb Chops (Chef Sofia, 100%) #25 Margherita Pizza (Chef Léa, 100%) #27 Beef Burger (Chef Ben, 100%) #28 Pasta Carbonara (Chef Marco, 100%) #29 Lamb Chops (Chef Amir, 100%) #30 Pasta Carbonara	#20 Caesar Salad (Chef Yuki, Done 3s ago) #24 Vegetable Stir-fry (Chef Amir, Done 5s ago) #23 Chocolate Lava Cake (Chef Carlos, Done 4s ago) #22 Beef Burger (Chef Ben, Done 8s ago) #26 Grilled Salmon (Chef Marco, Done 5s ago) #19 Margherita Pizza (Chef Yuki, Done 8s ago)